Independence Day.
You made it.



You have reached full course independence. Use your team, tools, documentation and C# learning to direct the complete production pipeline.

One complete Action: all nine videos, the demonstrated work or engine equivalents, useful evidence, an individual explanation and a personal GitHub handoff.

Four steps / one working record

PLAN	BUILD	EXPLAIN	SUBMIT
Story, GDD, board, tasks	All 9 videos and work	Whole-project teach-back	Personal GitHub + receipt

Visual hat: the player's view. Logical hat: the actions and order. Data hat: values/state and connections. Move deliberately between them. Teams divide production tasks; everyone still understands all required parts.

Be ready

Open your personal GitHub and engine/IDE. Check edit, run, save, commit, push and access. Keep the work on an approved data drive with space. Confirm the deadline in Canvas. This guide and form do not record attendance or submit work for you.

Plan the game. Make it visible.

Required reading: Attention First, Chapter 3, pp. 37-41. In your project document use **Pipelines** as section headings in place of Chapters. Name the sequence from the first idea to final handoff. For each Pipeline record its goal, input, tasks/owner, finished output and test/evidence. Show how its output begins the next Pipeline.

Example names: Pipeline 01 / Story and player plan; Pipeline 02 / Scene and assets; Pipeline 03 / Behavior and C#; Pipeline 04 / Test and release. Adapt these names to your project. Keep reading notes and an example of how you applied the assigned pages.

Continue a VOID RUNNERS contract, improve a previous idea or create your own. Keep the scope small enough to complete, test and explain.

1 / Story and player

Who is the player? What do they want? Where are they, and why does it matter? This is the backstory behind the game.

2 / Actions and data

List what the player does. Name the values/state and objects those actions need. A method is an action; a field/property stores data; a class describes objects and behavior.

3 / Prototype and whiteboard

Sketch the player's view in a slide/image editor or another accessible tool. Draw story -> actions -> data/objects -> scene -> expected result. Save a readable photo/export, names and contributions.

4 / Attention Accord and tasks

Agree what each person will investigate, capture and return, and when you will compare it. Keep Task | Owner | Status | Evidence. Name the lead and every contributor if grouped. Solo learners keep their own plan and a peer/instructor check.

YOUR WORKING NOTES	KEEP THE SAME RECORD
Backstory + player goal	_____
Actions / data / objects	_____
Board / prototype evidence	_____
Task / owner / check-in	_____

Use the editable HTML form for longer notes and embedded images. Save a filled copy before leaving the computer. Browser storage is only a draft.

Build / all nine videos.

Complete the demonstrated work, not just the watching. Record a short entry for each video. Other engines may translate the Unity portions to equivalent outcomes.

VIDEO	FOCUS / KEEP
1 / Game development introduction	Name the game you intend to make and the player's goal.
2 / Game development mindset	Explain how you will plan, investigate, build, test and communicate.
3 / Folders and project creation	Create or organize the project, documentation and evidence folders. Save a Git checkpoint.
4 / Documentation	Start your game design document: backstory, player, goal, actions, data and intended result.
5 / Introduction to Unity	Complete the demonstrated editor/project setup in Unity or your engine. Show and explain the equivalent project/scene.
6 / Import a custom player	Bring a player asset or suitable stand-in into your chosen engine. Explain its source, setup and role.
7 / Blender 3D	Follow the demonstrated asset workflow or explain and test your engine's equivalent asset workflow.
8 / Add textures	Apply the demonstrated visual treatment or equivalent material/texture workflow. Record sources.
9 / Final code	Complete and explain the tutorial behavior in your engine. Test the player's visible result and document errors honestly.

Open the complete mini-pipeline playlist

Investigate. Capture. Explain why.

One entry per video / use the form while working

PROMPT	WHAT TO RECORD
Source	Video/title, useful timestamp and official reference
Meaning	What is being taught? How does it help this project?
Attempt	What did you do? Which file, object, scene or behavior?
Result	What happened? Screenshot/path, output or exact error
Explanation	Why? What changed? What is the next test?

Translate the role / keep the same result

Unity Scene/GameObject/components become your engine's scene/level, nodes/actors/entities and behavior units. Map data, inputs, methods/events and output. Use your engine's actual API and language; explain its connection to the C# concepts you learned. Keep asset sources and dependencies. Do not paste Unity calls into another engine.

Required AI / one checked answer

Ask AI to explain a small relevant code/event excerpt, identify the data/actions/classes and suggest one test. Keep the prompt or supplied response, one claim, an official-source check, a test and your keep/change/reject decision.

No live AI? Verify this supplied response: "An object's behavior reads data and responds to an input or lifecycle event. A property change alone does not prove a working interaction; run a test and inspect the result. Engine APIs can differ." No paid or personal AI account is required.

Images and notes

Keep readable board, folders/setup, code/behavior and player-in-play screenshots. Add captions or paths that show what each proves. The portable HTML can embed PNG/JPEG/WebP images; keep original files if your notes link to them.

Explain the whole / 30 points.

Every learner clearly explains every required part, including parts contributed by peers. You may use your documentation and open the actual project. A team lead cannot speak for everyone.

COMMUNICATION	POINTS / WHAT TO SHOW
Purpose and plan	8 / Explain the player, backstory, goal, plan and why you chose this engine.
Technical understanding	8 / Open the actual project. Explain folders, scene/object structure, actions, data and relevant C# concepts or engine equivalents.
Evidence and questions	8 / Use notes/screenshots to explain what you tried, what happened, why, and what you will test next.
Whole-project handoff	6 / Explain all required parts, who did what, current status and the next-class/final plan. Answer follow-up questions clearly.

Gatekeeper review is active

The instructor checks notes/evidence, articulation, participation, collaboration/task follow-through and punctuality. All starred work must be present. A missing starred requirement may cap the assignment at 1 point under the syllabus. A form checkmark or receipt cannot verify understanding or attendance.

Late arrival

Lose 5 points for each completed five-minute interval late. Five minutes = 5 points; ten = 10; fifteen = 15. The final score stops at zero. The instructor records arrival and any approved adjustment.

Next-class plan

Write what is complete, remaining issues, next improvements, owners and the first test for the next class. Keep the relevant Discord exchange, question/feedback, your response and what changed.

Check the contract. Submit once.

* R1 / Complete the entire pipeline

Complete Videos 1-9 and their demonstrated work. Use Unity or translate the Unity portions to your chosen engine. Keep one learning entry per video and the editable project, player, assets and authored behavior.

* R2 / Read Attention First and name your Pipelines

Read Attention First, Chapter 3, pp. 37-41. In your project documentation use Pipelines as the section headings in place of Chapters. Name your project's Pipelines from start to finish. For each, give its goal, tasks/owner, input, finished output, test and evidence. Connect this sequence to your backstory, player goal, GDD and readable whiteboard; name contributions.

* R3 / Keep notes, screenshots and tests

Use the Action form throughout. Record what you learned, tried, observed and can explain. Include folders/setup, code or behavior, player-in-play, and expected/observed test evidence. Keep errors and the fix or next test.

* R4 / Show communication and task ownership

Keep concise Discord communications/peer feedback, a task list and your contribution. Groups name the lead, every member and who did what. Everyone must explain every required part using their own documentation. Solo learners keep their own tasks and a peer/instructor check.

* R5 / Write the next-class plan

State what is complete, remaining issues, next improvements, task owners and the first test for the next class. Include one checked AI answer or the supplied-response alternative and asset/source credits.

* R6 / Push and submit your personal GitHub

Keep the filled Action form, GDD, project source/assets/settings, board, screenshots, Discord record, tasks, next-class plan and receipt in your personal GitHub. Commit, push, test instructor access and submit your personal GitHub URL to Canvas the same class day. Link any team repository.

Final check: Save filled HTML and notes, including your reading application and named Pipelines. Keep GDD, source/scenes/assets/settings, board, screenshots/tests, Discord record, task list, next plan and source credits. Download the receipt into Evidence. Commit and push. Test instructor access and every referenced file/link. Submit the personal GitHub URL to the named Canvas Action; check confirmation. Link team work if used. No duplicate report.

30 points / due Monday, November 9, 2026 at 12:35 PM Pacific.

Open CS077 Canvas / submit

Open the working form | Create the receipt / final check